
Autonomous Parking Planning System

Sung-Han Tien (R14922137), Chih-Chuan Huang (R14944078), Po-An Chen (R14922152)
National Taiwan University
June 12, 2026

Abstract

This is an *implementation* project on autonomous parking motion planning. Parking is a difficult low-speed task that must respect a car’s nonholonomic kinematics, stay collision-free in a confined space, and often reverse to reach a precise final pose. We build an interactive simulator that, given a parking-lot geometry and a vehicle, computes a feasible, collision-free trajectory from a start pose into a target spot and animates the maneuver. Our scope is the planning problem itself: we assume the parking-lot geometry, the target spot, and any obstacles are given, and we do not address perception or detection. Within one shared vehicle model and coordinate system, we implement and compare three planners for an ordinary car, with methods tailored to the two most common maneuvers, parallel (curb-side) and perpendicular parking: (i) a Model Predictive Control (MPC) planner with maneuver-specific arc references for perpendicular and parallel parking; (ii) a Hybrid A* state-lattice planner with a Reeds-Shepp analytic shot that additionally handles user-placed obstacles; and (iii) a deep reinforcement-learning planner (SAC) that, once scaffolded by a Hybrid A* reference-path reward, demonstration seeding, and a reverse curriculum, parks reliably from the fixed start in both parking modes and partially generalizes to randomized scenes and single-obstacle layouts. We additionally extend Hybrid A* to the reverse path-planning of a vehicle with a single trailer — a different (tractor-trailer) vehicle model — reproducing a published method and further demonstrating it on a parallel parking scene not studied in the original work. We demonstrate each planner on representative scenarios, including cases with user-placed obstacles, and discuss the trade-offs between the geometric (MPC), search-based (Hybrid A*), and learning-based (deep RL) approaches.

Contributions of Each Member

- **Sung-Han Tien:** Base system / framework construction, MPC planner, and Hybrid A* planner.
- **Chih-Chuan Huang:** RL-related planner and the Reeds-Shepp analytic shot.
- **Po-An Chen:** Reproduction of the single-trailer truck-simulator paper (Hybrid A*-based reverse path planning).

Originality. This report does not contain any double assignment (e.g., a thesis or a project of another course) or any work completed before this semester.

Use of AI tools. We used the AI coding assistant Claude Code throughout the project. Our typical workflow was for the team to supply the original ideas together with existing open-source repositories and published work as references, discuss the design with the assistant, and then

have it carry out much of the implementation under our direction. All AI-generated code was reviewed, tested, and edited by the authors.

1 INTRODUCTION

Parking is one of the hardest everyday driving tasks, especially for novice drivers: it happens at low speed in tightly confined spaces, requires repeated forward and backward maneuvers, and must end at a precise final pose with very little room for error. Because it is both difficult and routine, parking assistance has become one of the most mature and widely deployed features of advanced driver-assistance systems (ADAS). In recent years many major automakers (e.g., Tesla, BMW, Mercedes-Benz, and Toyota) have shipped automatic-parking or summon products, which reflects both the practical demand and the technical maturity of the problem. From a research standpoint, parking is a small but representative motion-planning problem: it stresses the vehicle’s kinematic limits while leaving almost no margin for error, which makes it an ideal setting in which to implement and compare planning algorithms.

As noted above, this work focuses purely on the planning and computation side of parking: perception and detection problems (e.g., recognizing free spaces or sensing obstacles from camera or LiDAR data) are out of scope, and we assume the scene is already known. The planning problem alone is already challenging. A car is a *nonholonomic* system: it cannot move sideways, and its turning radius is bounded by the maximum steering angle and the wheelbase. A valid parking maneuver must therefore (i) respect these kinematic constraints, (ii) stay collision-free with respect to lane boundaries, parked cars, and obstacles, and (iii) often include one or more direction reversals (cusps) to fit into a narrow perpendicular or parallel spot. Adding a trailer makes the system even harder, because the trailer is only indirectly controlled and is unstable when reversing.

Existing parking planners broadly fall into three families [18]. *Geometric and control-based* methods (e.g., predefined arc references tracked by a model predictive controller) are smooth and fast, but are tailored to a specific geometry and have little obstacle awareness. *Search-based* methods such as Hybrid A* [14, 20] discretize the configuration space and search over motion primitives, handling obstacles and arbitrary geometry at a higher computational cost. *Learning-based* methods such as reinforcement learning learn a policy from interaction, trading guarantees for generality.

In this project we implement all three families inside a

single interactive simulator that shares one vehicle model, coordinate system, and parking-lot definition, so that the planners can be compared on equal footing. The system explicitly targets the two most common parking maneuvers, and we design methods tailored to each: parallel parking and perpendicular parking. For example, the MPC planner builds a different geometric reference for each maneuver (an S-curve for parallel parking versus a forward–reverse arc sequence for perpendicular parking), while Hybrid A* reaches both goal types through the same search formulation. Concretely, our contributions are:

- An **MPC** planner that builds and tracks geometric arc references for both **perpendicular** and **parallel** parking, with an early-stop (“park-then-align”) rule.
- A **Hybrid A*** planner with a **Reeds-Shepp analytic shot**, occupancy-grid and SAT collision checks, and support for an interactive, **user-placed obstacle**.
- A **deep RL** planner (SAC) trained via **reference-path potential reward** and reverse curriculum, with partial generalization to randomized scenes and limited obstacle settings.
- An extension of Hybrid A* to a **vehicle with a single trailer**, reproducing and extending the reverse path-planning method of [3] to a **parallel parking scene** not studied in the original work.
- An **interactive front end** (a settings window plus a pygame animation) that lets the user configure the scene and watch the planned maneuver.

The rest of the report is organized as follows. Section 2 describes the three single-vehicle planners; Section 3 presents the tractor–trailer extension; Section 4 presents the overall system design and implementation; Section 5 shows results on representative parking scenarios; and Section 6 concludes.

2 METHODS

The three planners in this section share the same vehicle model, coordinate system, and parking-lot geometry, so that they can be compared on equal footing. (The tractor–trailer extension in Section 3 uses a different vehicle model and is presented separately.) We model the car with the standard *bicycle kinematic model*, using the rear-axle center as the reference point. The state is (x, y, θ) and the controls are the speed v (negative for reverse) and the steering angle δ :

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \frac{v \tan \delta}{L_{wb}}, \quad (1)$$

where the wheelbase is $L_{wb} = 0.65 L$ (L is the car length) and the steering is bounded by $|\delta| \leq 35^\circ$, giving a minimum turning radius $R_{\min} = L_{wb} / \tan \delta_{\max}$. All geometry uses a right-handed world frame ($+x$ right, $+y$ up, in meters). The scene is a horizontal lane band plus a rectangular spot, above the lane for perpendicular parking and below it for parallel parking, and the car starts in the lane, heading $+x$, to the left of the spot. The following subsections describe these three planners.

2.1 MODEL PREDICTIVE CONTROL (MPC)

Our MPC planner works in two stages: it first builds a smooth geometric reference path from simple arcs and straight lines, and then tracks that reference with a receding-horizon MPC controller [19] that also penalizes driving the car body out of the drivable area. The speed profile (and thus all forward and reverse gear changes) is decided by the reference stage, so the controller only has to choose the steering angle.

For perpendicular parking, the reference has three phases: (1) *drive forward* along the lane to a stop point $x_{\text{stop}} = x_{\text{spot}} + R_{\min}$; (2) *reverse along a quarter-circle arc* of radius $R_{\min}$, turning the heading from 0 to -90° ; and (3) *reverse straight* into the spot. This is the familiar “pull past the spot, then back in” maneuver.

For parallel (curb-side) parking, the reference is an S-curve made of two equal-radius arcs. Let $\Delta y = \frac{1}{2}(w_{\text{lane}} + w_{\text{spot}})$ be the lateral shift needed to enter the curb-side spot, where w_{lane} and w_{spot} are the lane and spot widths. The arc angle is

$$\alpha = \arccos\left(1 - \frac{\Delta y}{2R_{\min}}\right), \quad (2)$$

which is feasible only when $\Delta y \leq 2R_{\min}$. After driving forward to $x_{\text{stop}} = x_{\text{goal}} + 2R_{\min} \sin \alpha$, the car makes a right-steer reverse arc ($\theta : 0 \rightarrow \alpha$) followed by a left-steer reverse arc ($\theta : \alpha \rightarrow 0$) that settles it parallel inside the spot.

The reference is tracked by a receding-horizon controller (horizon $N = 5$, step $dt = 0.05$ s). At each step it solves, with sequential least squares programming (SLSQP), for a short sequence of steering angles that minimizes

$$J = w_p e_p^2 + w_\theta e_\theta^2 + w_\delta \delta^2 + w_\delta \Delta \delta^2 + w_b P_b, \quad (3)$$

a weighted sum of position error e_p , heading error e_θ , steering effort, steering-rate (smoothness) $\Delta \delta$, and a large boundary-violation penalty P_b that keeps the car body inside the lane $\cup$ spot region. We use $(w_p, w_\theta, w_\delta, w_\delta, w_b) = (3.0, 1.5, 0.02, 0.30, 2000)$, with the steering bounded by $|\delta| \leq \delta_{\max}$. Only the first steering angle is applied (standard receding-horizon control), and the controller follows a look-ahead target along the reference.

A controller that follows the reference all the way to its end tends to keep reversing and clip the far edge of the spot. Instead, during the reverse phase we monitor the car body and apply a “park-then-align” early stop: as soon as the whole body is inside the spot and the heading is within 8° of the goal, we stop at the most-aligned in-spot pose recorded so far and discard any later drift. If a subsequent step would leave the drivable area but a valid parked pose has already been captured, we fall back to that pose and report success rather than a collision.

Finally, before simulating, the planner runs geometric feasibility checks (enough lane length beside the spot, the turning arc not clipping the road edge, and the car actually fitting in the spot) and records a status message;

during simulation the MPC additionally detects any real boundary violation of the car body.

2.2 HYBRID A*

The MPC planner above relies on a hand-built geometric reference and has no obstacle awareness, so it cannot adapt when the lane is blocked or the free space has an irregular shape. Hybrid A* removes both limitations: it searches directly in the vehicle’s configuration space and only ever expands kinematically feasible motions, so it can find a parking path around arbitrary obstacles.

Hybrid A* [14] is a variant of A* [16] that searches over the continuous pose (x, y, θ) while keeping the open and closed sets on a discretized grid (in our case 0.10m in position and 36 heading bins). From each node we expand a fixed set of *motion primitives*: forward and reverse, each combined with five steering angles $\{-\delta_{\max}, -\frac{1}{2}\delta_{\max}, 0, \frac{1}{2}\delta_{\max}, \delta_{\max}\}$, giving ten successors. Each primitive is a short arc rolled out with the same bicycle model and is discarded if any intermediate pose is invalid. Because the successors are generated by the real kinematics, every node in the tree is reachable by a drivable motion, the key difference from grid-based A*.

The edge cost is the primitive length plus penalties that encourage clean, natural maneuvers: reverse motion is slightly more expensive than forward, a direction change (cusp) adds a fixed penalty, and large or rapidly changing steering is penalized. The heuristic combines a holonomic term (Euclidean distance plus a heading term) with a *Reeds-Shepp (RS) lower bound* [15], the length of the shortest car-like path that ignores obstacles. The RS term is nonholonomic and therefore far more informative near the goal; we cache it on a coarse grid so that each lookup is effectively $O(1)$.

Pure lattice search wastes many expansions on fine alignment right at the goal. To avoid this we periodically attempt a *Reeds-Shepp analytic shot*, and always attempt one once the node is close to the goal: from the current node we compute the exact RS shortest path to the goal, and if every pose along it is collision-free and the final pose lies fully inside the spot, we accept it and finish immediately. This lets the search terminate with a smooth, exactly-aligned final maneuver.

Validity is checked against the full car footprint rather than a single point. The scene and obstacles are stored in an *occupancy grid*; for each pose we test all four car corners against the lane $\cup$ spot region and the rasterized obstacle cells, and additionally run an exact Separating-Axis-Theorem (SAT) test between the oriented car rectangle and each obstacle. The SAT test catches small obstacles that sit under the middle of a car edge without any corner landing in a blocked cell.

The goal is a rear-axle pose centered in the spot ($\theta = -90^\circ$ for perpendicular parking, $\theta = 0$ for parallel). The search stops when a node is within tolerance of the goal and the whole body is inside the spot. The raw lattice path is then smoothed by removing near-collinear way-

points whose shortcut segment is still collision-free, which shortens the path and removes small zig-zags introduced by the discretization.

2.3 REINFORCEMENT LEARNING

While geometric and search-based planners rely on explicit planning, RL can learn a closed-loop parking policy from interaction. In our implementation, however, we do not train the policy from scratch alone; instead, we scaffold learning with a Hybrid A* reference path, demonstration seeding, and a reverse curriculum. We first implement a tabular Q-learning baseline to expose the limitations of discrete-state learning, then develop a deep RL planner based on SAC. This section presents the RL environment, reward design, curriculum, and algorithm variants; the experimental comparison and final results are reported in Section 5.1.4.

2.3.1 TABULAR Q-LEARNING BASELINE

As a learning-based baseline, we implement a tabular Q-learning agent [9] on the same bicycle model. The continuous state (x, y, θ) is discretized into a grid ($\Delta_{xy} = 0.45$ m, $N_\theta = 12$ heading bins), and the action set is the Cartesian product of two gears $\{+1, -1\}$ and five steering angles $\{-\delta_{\max}, -\frac{1}{2}\delta_{\max}, 0, \frac{1}{2}\delta_{\max}, \delta_{\max}\}$, giving $|\mathcal{A}| = 10$ discrete actions. The reward uses potential-based shaping on goal distance and heading:

$$r = \underbrace{1.5 \Delta d + 0.4 \Delta \psi}_{\text{shaping}} - 0.05 + r_{\text{terminal}}, \quad (4)$$

with $r_{\text{goal}} = +25$ and $r_{\text{collision}} = -10$. Training uses ϵ -greedy exploration ($\epsilon : 0.95 \rightarrow 0.05$) with a reverse curriculum: 35% of episodes start near the goal so the agent learns terminal Q-values before attempting the full approach.

Q-learning trains for 40,000 episodes within a 30-second budget ($\alpha = 0.4$, $\gamma = 0.96$). Despite this budget, in our tests it never completed a valid parking maneuver from the simulator’s start pose. The failure is not a matter of tuning: the approach exhibits three fundamental limitations:

1. **Curse of dimensionality.** The SE(2) configuration space requires fine discretization for accurate maneuvers (± 0.35 m margin in a 2.5 m-wide spot), but the Q-table size grows as $O(N_x \cdot N_y \cdot N_\theta \cdot |\mathcal{A}|)$. At our resolution ($\sim 3,200$ cells $\times$ 10 actions), many states are visited too infrequently for convergence.
2. **Discretization artifacts.** The coarse heading resolution (30° bins) means the agent cannot represent the precise steering corrections needed to thread a 2.5 m-wide spot, and the grid-snapped actions produce zig-zag trajectories with unnecessary cusps.
3. **No generalization.** Each state is independent in the table; learning to park from one angle transfers nothing to nearby angles, so the agent must separately discover a policy for every entry trajectory [10].

These limitations motivate the use of *deep* RL with continuous state and action spaces, where a neural-network policy can generalize across nearby states and produce smooth, continuous steering commands.

2.3.2 DEEP RL ENVIRONMENT

We design a Gymnasium [11] environment that wraps the same bicycle model and parking-lot geometry but provides a continuous interface. The observation is a 23-dimensional vector:

$$o = (\Delta x, \Delta y, \sin \Delta\theta, \cos \Delta\theta, v, \delta, d_{\text{wall}}, \underbrace{o_1, \dots, o_{16}}_{4 \text{ nearest obstacles}}), \quad (5)$$

where $(\Delta x, \Delta y)$ is the goal position expressed *in the car's body frame*, $\Delta\theta$ is the heading error, v and δ are the current speed and steering, d_{wall} is the minimum distance from any car corner to the nearest boundary wall, and each obstacle feature is (dx, dy, w, h) in the ego frame. All features are normalized to roughly $[-1, 1]$ (positions by 10 m, v and δ by their maxima, d_{wall} capped at 3 m). Two details matter in practice. First, expressing the goal in the body frame makes the policy's input invariant to where in the world the maneuver happens. Second, d_{wall} treats the opening between the lane and the spot as *open space*, measuring the distance to the opening's two corner posts rather than to the shared edge, so that neither the observation nor the proximity penalty below reports "wall ahead" on the very threshold the car must drive through. The action is continuous: $a = (\delta_{\text{cmd}}, v_{\text{cmd}}) \in [-1, 1]^2$, mapped to physical steering and speed and held for one control period.

2.3.3 REWARD DESIGN

Dense shaping is essential at this horizon, but the *shape* of the potential matters more than its density. Our first design, following common practice [7], shaped on the Euclidean goal distance plus a spot-entrance waypoint bonus; every policy trained with it converged to hovering in the lane below the spot. The failure is structural: a perpendicular maneuver must first drive *past* the spot to set up the reverse arc, and every meter of that overshoot *increases* the Euclidean goal distance, so the shaping punishes exactly the behavior the task requires. Replacing the Euclidean distance with the Reeds-Shepp length [15] fixes the overshoot but introduces a subtler flaw: RS ignores boundaries, so its value is nearly flat along the lane segment between the spot and the gear-switch point (its optimal paths cut through the lane wall), and the policy again stalls exactly where it must commit.

We therefore shape on the distance-to-go *along the Hybrid A* reference path* (Section 2.3.4). Let $\hat{s} = (\hat{p}, \hat{\theta})$ be the reference pose closest to the state $s = (p, \theta)$ under the metric $\|p - \hat{p}\|^2 + 2\Delta\theta^2$, and let $\ell_{\text{rem}}(\hat{s})$ be the arc length remaining from $\hat{s}$ to the goal. The potential is

$$\Phi(s) = \ell_{\text{rem}}(\hat{s}) + 2.5 \|p - \hat{p}\| + |\Delta\theta| + 0.8 \min(d_{\text{RS}}(s), 5). \quad (6)$$

The deviation weights are deliberately large: with weak weights the projection itself creates an exploit — cutting inside a curve advances $\hat{s}$ faster than the deviation term costs — and trained policies arrived at the spot over-rotated and wedged against its side. The capped RS term restores a sharp alignment gradient near the goal, where small lateral or heading errors can require substantial car-like correction, while reducing to a constant in the far field. For each fixed scene, the Hybrid A* reference path defines a state-dependent progress signal that decreases along the intended maneuver and vanishes near the goal. We therefore use it as a dense shaping term to guide the policy toward the planned parking behavior, rather than as a strict guarantee of policy invariance across all randomized scenes. The full reward is

$$r = 2r_{\Phi} + r_{\text{prox}} + r_{\text{term}} - 0.02 - 0.01 \frac{|\delta|}{\delta_{\text{max}}} - 0.05 \frac{|\Delta\delta|}{\delta_{\text{max}}} - 0.1 [\text{cusp}]. \quad (7)$$

The proximity penalty

$$r_{\text{prox}} = - \frac{\max(0, m - d_{\text{wall}})}{m} (0.3 + 0.7 |v|/v_{\text{max}}) \quad (8)$$

gives a gradient before collision, with two corrections to our original design. The margin $m = 0.3\text{ m}$ is set *below* the in-spot lateral clearance (0.35 m at the default sizes), so a correctly parked pose costs nothing; with our original $m = 0.8\text{ m}$ the penalty taxed every step spent inside the spot. And d_{wall} is the opening-aware wall distance of the previous section; with the opening treated as a wall, the penalty formed a barrier across the spot threshold, and trained policies backed in, aligned perfectly, and stalled two meters short of the goal. Terminal rewards are speed-scaled: $r_{\text{success}} = 100 - 10 |v|/v_{\text{max}}$ and $r_{\text{collision}} = -(50 + 25 |v|/v_{\text{max}})$.

2.3.4 REFERENCE PATH AND DEMONSTRATIONS

The reference path is planned once per scene by Hybrid A* (Section 2.2) from the fixed start pose to the goal, but *not* with the true vehicle: the planner tries a ladder of progressively smaller safety margins, inflating the car body by $(+0.50, +0.30)$, $(+0.35, +0.20)$, or $(+0.15, +0.10)\text{ m}$ with the steering capped at 80/85/90% of δ_{max} , and keeps the first feasible plan (the default scene resolves to the $(+0.35, +0.20)\text{ m}$ / 85% tier). Raw A* solutions hug the boundary with $\sim 3\text{ cm}$ of clearance at the minimum turning radius; no feedback controller (or policy) can track such a reference without colliding, because no clearance or curvature authority is left for corrections. The margin-planned reference keeps both.

Demonstrations are produced by a gear-aware *Pure Pursuit* controller — a classic path tracker that, at each step, steers the rear axle toward a look-ahead point a fixed distance ahead on the path (under the same bicycle model) — which tracks the densified reference *through the environment itself*, starting from the fixed start with small perturbations. The path is split at cusps into single-gear

segments, and each cusp must be reached within 0.12 m before the next segment starts: entering the reverse arc a few degrees under-rotated is unrecoverable, since the arc leaves no spare steering authority. For reverse segments the lookahead target is mirrored into the frame of a virtual forward-driving car *and the resulting steering angle is negated*, since $\theta = v \tan \delta / L_{wb}$ flips sign with v , a mirrored target alone (a bug in our first implementation, and in our first residual-RL baseline) steers the car off the path on every reverse segment. The corrected controller parks in 30/30 demonstration episodes on the default scene (on the tighter randomized scenes of Section 2.3.7 it succeeds less often, so only successful episodes are admitted to the buffer). The demonstration transitions seed the SAC replay buffer, and the actor mean is behavior-cloned to mitigate initial exploration costs while avoiding the compounding errors of pure imitation [8], before RL begins.

2.3.5 CURRICULUM AND START DISTRIBUTION

Exploration is organized as a *reverse curriculum along the reference path*: a start pose is sampled at arc-length fraction $t \in [t_{lo}, 0.9]$ of the reference (plus pose noise), and t_{lo} shrinks over five stages ($0.6 \rightarrow 0.3 \rightarrow 0 \rightarrow 0 \rightarrow 0$), so the agent first learns the final reverse-in, then the gear switch, and finally the full approach; the last two stages additionally randomize the scene and add an obstacle (Section 2.3.7). Each stage also draws a share of episodes from the exact fixed start (10/20/50/40/40%), and the randomized stages mix in random in-lane poses (20/10%) for robustness. Starts closer than 0.8 m to the goal are rejected: with the rear axle nearly against the spot’s back wall, any reverse step collides, and such unwinnable episodes pollute both training and evaluation. Stage promotion requires a 70/60/60/50/40% evaluation success rate. Finally, because mean evaluation reward does not track the metric the simulator deploys on, checkpoints are selected by the *success rate from the fixed start*, measured by a dedicated callback.

2.3.6 ALGORITHMS

We compare four standard deep RL algorithms on the perpendicular task:

1. **SAC** [4]: Soft Actor-Critic with automatic entropy tuning ($\text{lr} = 3 \times 10^{-4}$, replay buffer 10^6 , batch 256, $\gamma = 0.99$, two 256×256 hidden layers).
2. **TD3** [5]: Twin Delayed DDPG with Gaussian exploration noise ($\sigma = 0.1$).
3. **PPO** [6]: Proximal Policy Optimization, on-policy with GAE ($\lambda = 0.95$).
4. **SAC+HER**: SAC with Hindsight Experience Replay [17] (goal-conditioned, `MultiInputPolicy`, 4 future goals).

We additionally implement a **residual RL** [13] variant in which the executed action is $a = \text{clip}(a_{\text{base}} + 0.15 a_{\pi})$, where a_{base} comes from the pure-pursuit base controller of Section 2.3.4 and a_{π} is a SAC correction. Two details

are required to make this work: the policy mean head is zero-initialized, so the deterministic policy starts as *exactly* the base controller and RL only refines it; and the entropy coefficient is initialized low ($\alpha_0 = 0.05$), because with the default initialization ($\alpha \approx 1$) the first actor updates maximize entropy against a still-meaningless critic and destroy the already-successful base behavior within a few thousand steps.

2.3.7 SCENE RANDOMIZATION, OBSTACLES, AND PARALLEL PARKING

To move beyond a single scene, we extend the curriculum with two domain-randomization stages. A pool of 26 scene combinations (lane width, spot size, car size) is sampled from the ranges exposed by the simulator’s settings sliders, anchored on the UI and `config.py` defaults and pre-filtered by simple clearance rules. An *obstacle stage* additionally drops one 0.9 m box at one of 5-6 predefined, lot-relative positions (approach, under the spot, past the spot). Crucially, a (scene, obstacle) world is admitted to training and evaluation *only if Hybrid A* finds a collision-free path in it*, so every episode is parkable by construction; this feasibility check reuses the reference-path computation and is cached on disk.

Fine-tuning the trained fixed-start policy on these stages initially failed by *catastrophic forgetting*: three runs lost the base behavior within 15k steps, regardless of learning rate. The cause is structural, a from-scratch run that reaches the randomization stage replays hundreds of thousands of base-scene transitions from its buffer, whereas a warm start begins with an *empty* buffer, so out-of-distribution scenes immediately swamp the critic. The cure is to restore that anchor: before fine-tuning we prefill the buffer with 30k base-scene transitions collected by the warm policy itself, and 40% of randomized-stage episodes rehearse the base scene.

For *parallel parking*, from-scratch SAC failed in three distinct ways (twice by entropy-coefficient explosion, while the task is unsolved the policy’s entropy sits below any target, automatic tuning grows α without bound until the actor emits NaNs, and once by degenerate convergence to dithering in place); the in-spot multi-point straightening makes exploration far harder than the perpendicular reverse-in. We therefore deploy the residual formulation above. Two additions make it reliable: the final reference segment is extended ~ 1 m beyond the goal along the goal heading, so the pure-pursuit base straightens *through* the endpoint instead of stopping tilted just outside the success window; and the fixed-start evaluation callback also runs at step 0, when the zero-initialized residual equals the base controller, so the deployed checkpoint is *provably at least as good as the base* no matter how training proceeds. In our run the trained policy surpassed this gate after 260k steps.

3 HYBRID A* FOR A VEHICLE WITH A SINGLE TRAILER

This section reproduces and extends the Hybrid A*-based reverse path-planning method proposed by Cao et al. [3]. The original paper addresses the problem of planning a kinematically feasible, collision-free reverse parking path for a tractor-trailer system, with two key contributions: (i) an inverse-kinematics reparameterization that implicitly prevents jackknifing by constraining the search to physically realizable hitch angles, and (ii) a simplified collision-avoidance scheme that inflates static obstacles by the vehicle half-width so that only system center-lines need to be checked at runtime. We implement this method within our shared simulator, verify that it reproduces the paper’s perpendicular-docking results, and additionally apply it to a parallel-parking scene not studied in the original work.

3.1 WHY TRUCK-TRAILER PARKING DIFFERS FROM SINGLE-VEHICLE PARKING

A passenger car is controlled by a single pair of inputs (δ_f, V_R) and its configuration space is 3-dimensional (x, y, θ). Adding a trailer introduces a second rigid body connected at a hitch joint, expanding the state to six dimensions and creating a qualitatively different planning problem in the following respects.

UNINTUITIVE REVERSE DYNAMICS. When a car reverses, the front axle follows the rear axle and the heading change is straightforward. A trailer, by contrast, rotates *opposite* to the tractor when reversing: turning the steering wheel left causes the trailer to swing right. The coupling is non-linear and depends on the current hitch angle $\Delta\psi$, so the required steering input to achieve a given trailer heading change varies continuously throughout the maneuver.

JACKKNIFE INSTABILITY. If $|\Delta\psi|$ reaches 90° the kinematic equations become singular and the system loses controllability — a jackknife event. This places a hard state constraint on any valid path that does not exist for single-vehicle planning; a planner that ignores it may find a nominally connected path that still causes a jackknife, as illustrated in Figure 1.

LARGER FOOTPRINT AND TIGHTER COLLISION GEOMETRY. A tractor-trailer occupies a much larger area than a car and the two units can swing independently, so the swept envelope during a maneuver is non-convex and hard to bound tightly. Our implementation must check eight corners (four per unit) at every search node rather than the four corners of a single car.

LARGER MINIMUM-SPACE REQUIREMENT. The combined length of tractor and trailer ($l_{\text{truck}} + L_H + L_T + l_{\text{trail}} \approx 13.5\text{m}$ for the parameters in Table 1) means that a viable parking slot must be substantially longer than for a passenger car. The minimum turning radius constraint also

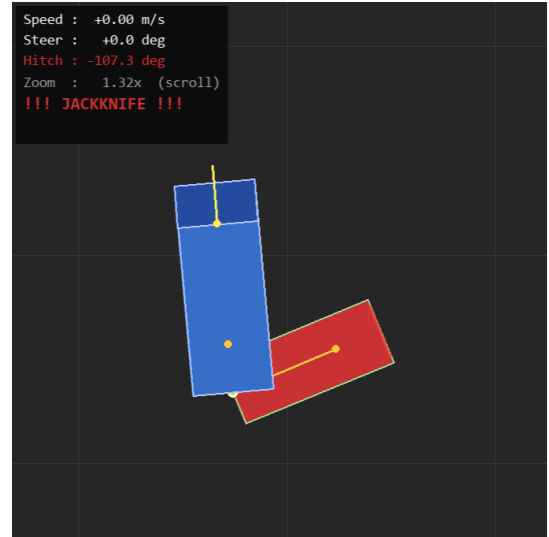


Figure 1: A jackknife event in our simulator: the hitch angle has reached -107° , far past the 90° singularity, and the trailer (shown in red) has folded back against the truck body.

becomes more restrictive because the effective wheelbase is longer.

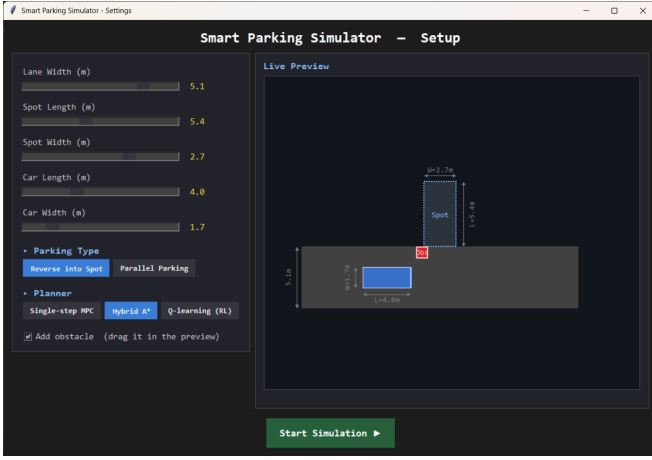
These properties make a trailer-aware planner necessary: the MPC arc-reference approach (Section 2.1) is not readily adaptable because the reference geometry changes with $\Delta\psi$, and the single-car Hybrid A* (Section 2.2) does not model the trailer at all.

3.2 TRACTOR-TRAILER KINEMATIC MODEL

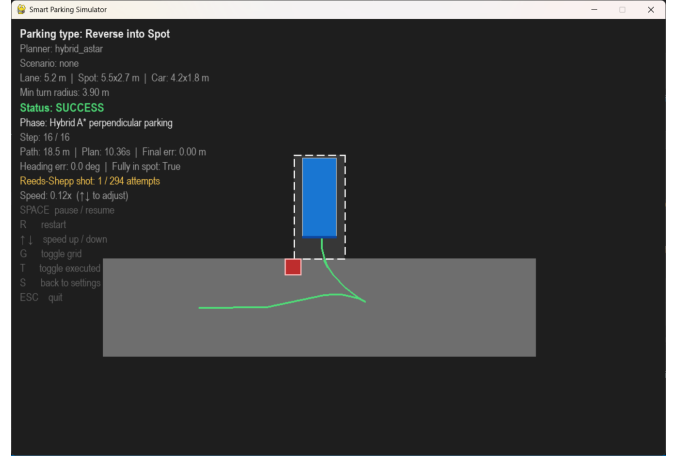
We adopt the kinematic vehicle-trailer model of the reference paper [3] directly. The system state is the 6-tuple $(x_R, y_R, \psi_1, x_T, y_T, \psi_2)$, where (x_R, y_R, ψ_1) is the rear-axle pose of the truck and (x_T, y_T, ψ_2) is the pose of the trailer axle. The *hitch angle* $\Delta\psi = \psi_1 - \psi_2$, normalized to $(-\pi, \pi]$, measures the articulation between the two units; if $|\Delta\psi|$ reaches 90° the system enters an uncontrollable *jackknife* state. Control inputs are the front-wheel steering angle δ_f and the rear-axle speed V_R (positive forward). We integrate the forward kinematics using a fourth-order Runge-Kutta (RK4) scheme at a fixed sub-step of $\Delta t = 0.2\text{ s}$ during planning, giving five sub-steps per node expansion ($N_{\text{sub}} = 5$, total horizon $T = 1.0\text{ s}$). The vehicle parameters used in our implementation are listed in Table 1 and illustrated in Figure 2.

3.3 TRAILER-CENTRIC STATE REPRESENTATION AND INVERSE KINEMATICS

Directly parameterizing the search over δ_f is inconvenient: at large $|\Delta\psi|$ the same δ_f produces very different trailer yaw rates. Following the reference paper we instead sample a *virtual trailer steering angle* δ_T at each node expansion and recover the physical controls (δ_f, V_R) via inverse kinematics. Both expressions become singular when the

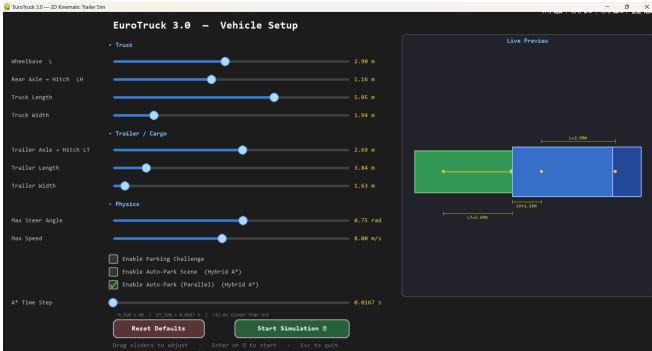


(a) Settings window with live preview.

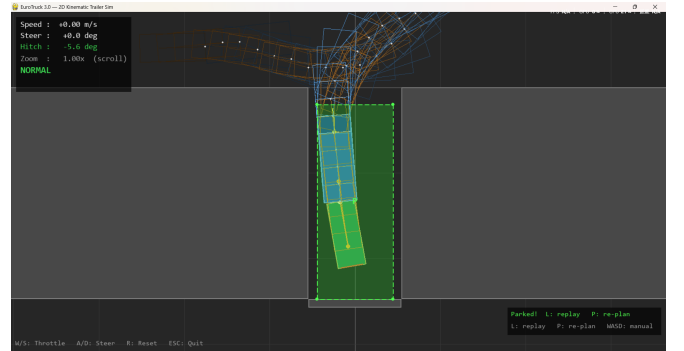


(b) Simulation of the planned maneuver.

Figure 3: The single-vehicle simulator: the configuration screen (left) and the animated parking maneuver (right).



(a) Configuration screen with truck/trailer parameters.



(b) Simulation of the auto-park maneuver.

Figure 4: The truck-trailer simulator: the configuration screen (left) and the animated parking maneuver (right).

4.1 SINGLE-VEHICLE SIMULATOR

The single-vehicle part runs in two sequential phases and loops between them. In the *settings phase*, a Tkinter window (Figure 3, left) presents sliders for the car and parking-lot dimensions, toggle buttons for the parking type (perpendicular or parallel) and the planner, an optional draggable obstacle for Hybrid A*, and a live preview canvas. On confirm it returns two small dataclasses, a **ParkingConfig** and a **CarConfig**, which are the only data passed between phases. In the *simulation phase* (Figure 3, right), the planner is invoked and the resulting maneuver is animated; pressing a key returns to the settings phase to try a different configuration.

The two phases are decoupled by a single entry point, `plan_trajectory()`, which inspects the configuration and dispatches to the chosen backend (MPC, Hybrid A*, or RL) behind a uniform interface. Every planner returns the same list of waypoints, so the simulation and rendering code is identical regardless of which planner produced the path. This makes the planners directly interchangeable and is what lets us compare them on equal footing. A small set of shared modules builds the world-space lane and spot rectangles and the car footprint from the two

configs, using a right-handed frame ($+x$ right, $+y$ up) with the rear-axle center as the reference point, and renders the scene, the animated car, and optional overlays through an on-screen HUD.

4.2 TRUCK-TRAILER SIMULATOR

The truck-trailer part follows the same configure-then-simulate pipeline, but instantiates it with the articulated vehicle model. It supports both a perpendicular reverse-parking scene and a parallel (roadside) parking scene. Unlike the single-vehicle part, the state is six-dimensional, the truck rear-axle pose (x_R, y_R, ψ_1) plus the trailer pose (x_T, y_T, ψ_2) , with the hitch angle $\Delta\psi = \psi_1 - \psi_2$ as the key coupling variable. The user can either drive manually or invoke an auto-park mode that executes a planned path.

Because the articulated vehicle has many more degrees of design freedom, its configuration screen (Figure 4, left) exposes a richer parameter set, grouped into three sections with a live preview that redraws the vehicle as the values change. The *truck geometry* controls the wheelbase L (front-to-rear axle distance), the rear-axle-to-hitch length L_H , and the truck body length and width. The *trailer geometry* controls the trailer-axle-to-hitch length L_T and

the trailer body length and width. Together, L , L_H , and L_T define the articulated kinematics: they govern how strongly the trailer reacts to steering and how easily the system can jackknife, while the body sizes set the swept footprint and therefore the smallest slot the vehicle can park in. The *physics* section sets the maximum steering angle (which bounds the minimum turning radius) and the maximum speed. When an auto-park mode is enabled, an additional A* time-step slider controls the planner’s integration step, trading planning resolution against search speed.

The auto-park planner (Figure 4, right) is a Hybrid A* search over a four-dimensional trailer-centric state $(x_T, y_T, \psi_2, \Delta\psi)$. It searches in a virtual trailer control space (δ_T, V_T) and uses an inverse-kinematics mapping to recover the physical truck steering and speed (δ_f, V_R) ; this reparameterization (from the reproduced paper) implicitly bounds the hitch angle and prevents jackknifing. Because this search is heavier than the single-car one, it runs in a background thread while the main loop stays responsive, and the motion is integrated with RK4 at the frame rate, so the executed path reproduces the planned one with no integration mismatch. Collision checking uses eight corners (four per body) against the scene walls, reflecting the larger, non-convex swept envelope of the articulated vehicle.

5 RESULTS

We evaluate the planners qualitatively on representative scenarios of both parking modes, with the difficulty increasing from one scenario to the next, and discuss the trade-offs between the geometric, search-based, and learning-based approaches.

5.1 SINGLE-VEHICLE SIMULATION

5.1.1 MPC IN BOTH PARKING MODES

In a comfortably sized scene (lane width 5.0 m, spot 5.8×2.8 m, car 3.7×1.7 m), the MPC planner parks the car successfully in both the perpendicular and parallel modes, following its geometric arc reference into the spot with a smooth, low-cusp trajectory (Figure 5).

5.1.2 MPC LIMITATIONS AND HYBRID A*

As the scene tightens, a narrower lane (4.7 m), a tighter spot (5.6×2.5 m), and a larger car (4.2×1.9 m), the MPC planner’s fixed geometric reference no longer fits: tracking it drives part of the car body out of the drivable area, so the maneuver is reported as a collision in both modes (Figure 6, top row). Under the *same* configuration, Hybrid A* still finds a feasible, collision-free path for both modes (Figure 6, bottom row). This directly substantiates the limitation noted in Section 2.1, the MPC reference is brittle in narrow spaces — and shows that search-based planning generalizes to harder geometry at the cost of a less regular path.

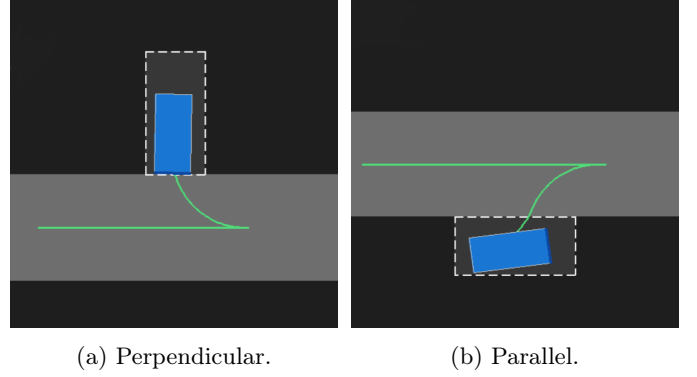


Figure 5: MPC parks successfully in both modes in a roomy scene.

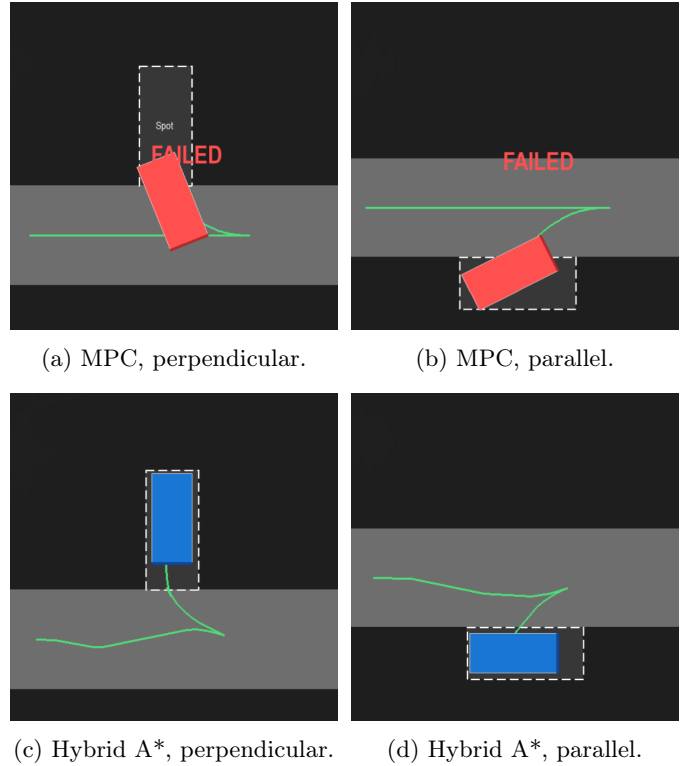


Figure 6: In a tighter scene, MPC fails (top) while Hybrid A* succeeds (bottom) under the same configuration, for both parking modes.

5.1.3 OBSTACLE HANDLING WITH HYBRID A*

Hybrid A* is also the only planner that reasons about obstacles. In an even narrower lane (4.1 m, spot 5.5×2.7 m, car 4.0×1.8 m), we compare the planned perpendicular path with and without a user-placed obstacle (Figure 7). Without the obstacle the planner takes a direct reverse-in path; with the obstacle dropped into the approach, it automatically re-routes around it and still parks fully inside the spot. This demonstrates the value of the occupancy-grid plus SAT collision model from Section 2.2.

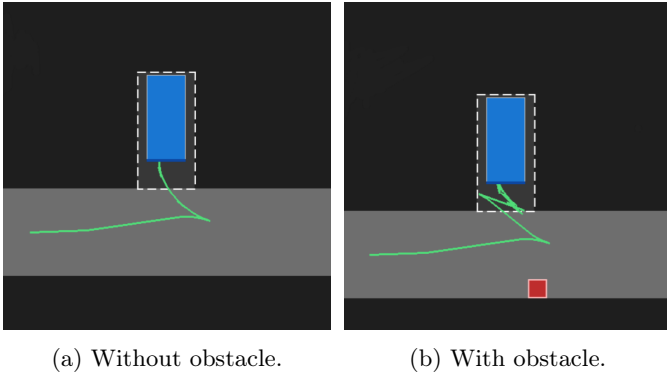


Figure 7: Hybrid A* re-routes around a user-placed obstacle while still parking in the spot (perpendicular).

5.1.4 REINFORCEMENT LEARNING

BASILINE ALGORITHM COMPARISON. Table 2 compares the four algorithms of Section 2.3.6 under our *initial* environment design (Euclidean shaping and radial start curriculum), each trained for 10^6 timesteps. Under this protocol, SAC performs best among the tested algorithms, with the highest overall success rate and the lowest collision rate. This suggests that maximum-entropy exploration is better matched to the multi-modal parking action space than the alternatives tested here. However, the fixed-start column is the decisive result: every algorithm scores 0% from the simulator’s fixed start pose, which is the case used by the interactive system. All successes therefore came from curriculum starts near the goal, showing that the initial environment did not solve the actual deployment setting. TD3, PPO, and SAC+HER were less effective under this setup. A plausible explanation is that Gaussian exploration, on-policy sampling, and HER relabeling respectively do not directly address the long-horizon, multi-phase structure of the parking maneuver.

Table 2: RL algorithm comparison under the *initial* environment (perpendicular parking, 10^6 steps, 100-episode evaluation; random starts drawn from its near-goal-biased curriculum).

Algorithm	Success	Fixed	Random	Collision
SAC	32%	0%	64%	2%
SAC+HER	18%	0%	36%	10%
PPO	9%	0%	18%	19%
TD3	0%	0%	0%	0%

DIAGNOSIS. Rollout probes traced this failure to the environment rather than the algorithms. The Euclidean potential punishes the forward overshoot, so policies hovered below the spot and timed out (99/100 evaluation episodes); an in-spot state bonus paid more for loitering than the terminal reward paid for finishing; the proximity penalty’s margin exceeded the in-spot clearance, taxing every correctly parked step, and its wall model placed a

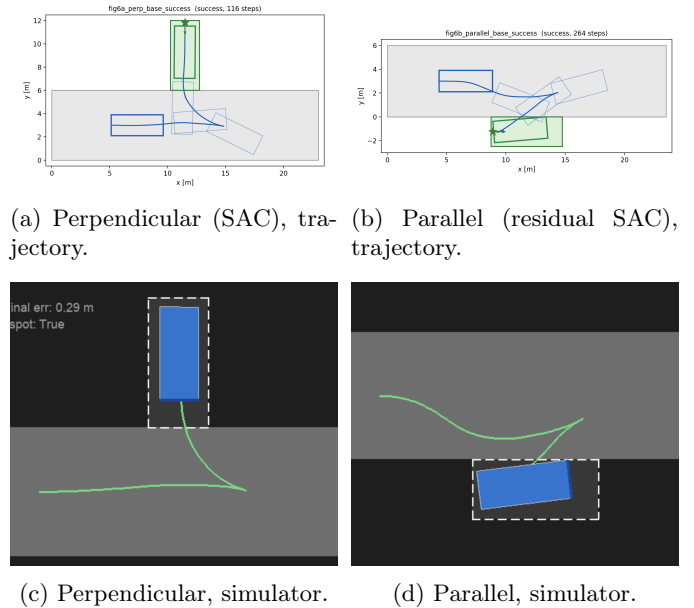


Figure 8: The deployed RL policies park successfully in both modes on the base scene from the fixed start. Top row (a, b): rendered rear-axle trajectories of the deterministic rollouts (blue path, car footprints at intervals, goal star); bottom row (c, d): the same maneuvers executed in the interactive simulator.

penalty barrier across the lane-spot opening, so policies backed in, aligned, and stalled short of the goal; and the original demonstration collector never produced a reverse action (its speed command was a norm), so the seeded replay buffer contained no successful parking at all. The corrections are the methods of Sections 2.3.3–2.3.6.

FINAL RESULTS. Table 3 evaluates the overhauled system with 100 episodes per method: half from the exact fixed start and half from the hardest start distribution (poses along the full reference path and random in-lane poses). SAC trained in the final environment parks from the fixed start in 100% of episodes and from randomized starts in 88%, with *zero* collisions, every residual failure is a time-out, not a crash. Within the evaluated perpendicular no-obstacle protocol, the learned SAC policy outperforms the pure-pursuit reference tracker it was seeded from, suggesting that it learned corrections beyond simply imitating its scaffold. The residual variant also holds 100% on the fixed start but adds less on randomized starts, limited by its ± 0.15 correction authority. From the exact fixed start the final SAC policy parks with 0.24 m position and 0.5° heading error in 72 control steps with a single gear switch, and succeeds in 20/20 episodes under start perturbations of ± 0.5 m and $\pm 8^\circ$. Figure 8 shows the two deployed policies on the base scene: the perpendicular model executes the standard “pull past, then reverse in” maneuver with a single cusp, and the parallel model the S-curve entry followed by an in-spot straightening wiggle.

Table 3: Final evaluation under the overhauled environment (perpendicular parking, no obstacles; 100 episodes per method: 50 from the fixed start, 50 from randomized full-path / in-lane starts). The last row repeats the initial-environment SAC for reference (its random column uses the easier near-goal protocol of Table 2).

Method	Overall	Fixed	Random	Collision
SAC (final env)	94%	100%	88%	0%
Pure pursuit (no RL)	87%	100%	74%	13%
Residual SAC	82%	100%	64%	10%
SAC (initial env)	32%	0%	64%	2%

TRAINING DYNAMICS. Two practical lessons generalize. First, learning looks deceptively stalled: success from near-goal starts plateaus around 30–40% for the first $\sim 10^5$ steps while the critic slowly acquires action discrimination, a single control step changes the state so little that the action-dependent part of Q is only a few percent of its magnitude, and the full fixed-start maneuver is solved only after $\sim 4\text{--}5 \times 10^5$ steps (about two hours on CPU). We discarded several ultimately-viable runs by judging them at 60k steps. Second, checkpoint selection must use the deployment metric: the mean-reward-best and fixed-start-best checkpoints differ, and only the latter is reliable in the simulator.

GENERALIZATION: RANDOMIZED SCENES AND OBSTACLES. Table 4 evaluates the two deployed policies after the expansion of Section 2.3.7 (the perpendicular model additionally received an obstacle-focused fine-tuning round, the same reverse curriculum restricted to base-scene obstacle worlds). It keeps a *perfect* 20/20 on the base scene while acquiring partial competence on worlds it could not handle before: 15/30 when both the scene dimensions and the start pose are randomized, and 16/30 when one feasibility-checked obstacle is also placed in the lane. Its failures are mostly conservative: in 60 randomized episodes, it collided only 3 times, otherwise timing out at a stand-still. This suggests that the dense proximity penalty encourages safe stopping behavior, although it does not by itself solve obstacle detouring. Figure 9 shows the characteristic behavior with a user-placed obstacle on the base scene.

PARALLEL PARKING. The deployed parallel policy is the residual model of Section 2.3.7. It parks 20/20 from the fixed start with 0.08m position and 4.8° heading error, executing the classic forward-reverse S-curve followed by an in-spot straightening wiggle. Generalization is weaker than for the perpendicular model: 10/30 on randomized scenes, with collisions rather than timeouts as the dominant failure, when the pure-pursuit base strays from its reference in an unfamiliar scene, the ± 0.15 residual authority is too small to recover. The contrast between the two maneuvers is itself a finding: the perpendicular task was solvable by pure SAC, while parallel parking defeated three

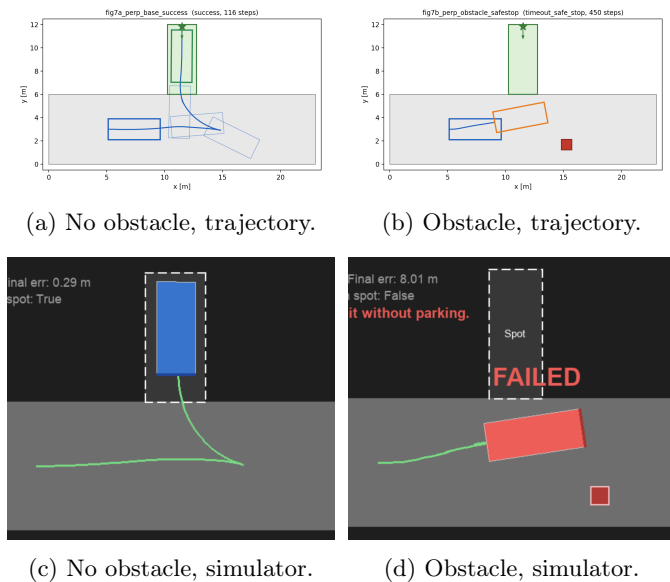


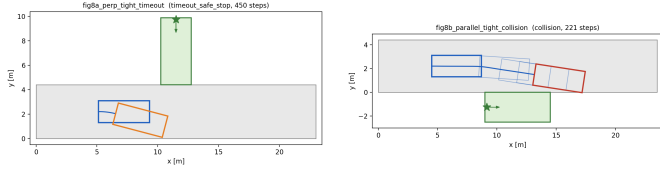
Figure 9: The perpendicular RL policy on the base scene without (left column) and with (right column) a user-placed obstacle in the lane; rendered trajectories on top (a, b), the simulator view below (c, d). Without the obstacle the policy parks; from the *fixed start* it does not yet find a way around the obstacle and instead stops safely in front of it (its obstacle competence is currently limited to the randomized-start protocol of Table 4).

from-scratch runs (entropy-coefficient explosion twice, degenerate freezing once) and was only delivered by constraining RL to corrections around a classical controller.

Table 4: Deployed-model evaluation after the expansion (deterministic policies; collision counts in parentheses). Randomized rows draw both the scene dimensions and the start pose at random from the training pool; the obstacle row additionally places one feasibility-checked box in the lane.

Evaluation	Perp. (SAC)	Par. (residual)
Base scene, fixed start	20/20 (0)	20/20 (0)
Randomized scenes	15/30 (2)	10/30 (14)
Randomized + obstacle	16/30 (1)	—
Tight 4.4 m lane, fixed	0/10 (0)	0/10 (10)

LIMITATIONS. Three limitations of the RL planners deserve emphasis. First, *generalization across scene sizes is partial*: roughly half of the randomized scenes for the perpendicular model and a third for the parallel one. The tightest scenes (e.g. a 4.4m lane with the default car) remain unsolved by both models, there, even the margin-planned reference path retains only centimeters of clearance, so neither the demonstrations nor the reward potential offer a corridor the policy can actually follow; scene-conditioned observations or per-scene fine-tuning are natural remedies. Figure 10 shows the two characteristic failure modes in such a scene: the perpendicular policy refuses to



(a) Perp.: safe timeout, traj. (b) Parallel: collision, traj.



(c) Perp., simulator. (d) Parallel, simulator.

Figure 10: Characteristic RL failures in the tight 4.4-m-lane scene (spot 5.5×2.5 m, car 4.2×1.8 m), shown as rendered trajectories (a, b) and in the simulator (c, d): the perpendicular policy fails *safely* by stopping in the lane, while the parallel one clips the spot during the S-curve.

commit and stops in the lane (a safe timeout), while the parallel policy’s base controller strays off its reference during the S-curve and the limited residual authority cannot recover, ending in a collision.

Second, the obstacle capability is narrow by design: one box of fixed size, drawn from predefined positions, with feasibility guaranteed by a Hybrid A* check — the policy reacts to the obstacle it observes but has never faced multiple or moving obstacles. Moreover, its obstacle competence lives on the randomized-start protocol: from the *fixed start* it does not yet complete a maneuver around an obstacle, even after a dedicated fine-tuning round (it learned the near-goal half of the detour but not the full approach), and instead fails safely by stopping (Figure 9). Third, the training process itself proved fragile in ways that required engineering rather than tuning: warm-started fine-tuning forgets catastrophically without a replay anchor, and SAC’s automatic entropy tuning diverges on long-unsolved tasks; both cures (buffer prefill, fixed or floored entropy coefficient, and deployment-metric checkpoint gating) are described in Sections 2.3.6–2.3.7 and are, in our view, as important to the final result as the reward design.

5.1.5 CROSS-PLANNER BENCHMARK

Finally, Figure 11 compares the planners on the evaluator’s ten standard cases: two parking modes crossed with five scenarios, including one obstacle-free case and four named obstacle layouts (entry blocker, tight 3.8 m lane, pillar near the entry, and parked cars in adjacent spots), with one deterministic run per case on the default scene. Hybrid A*

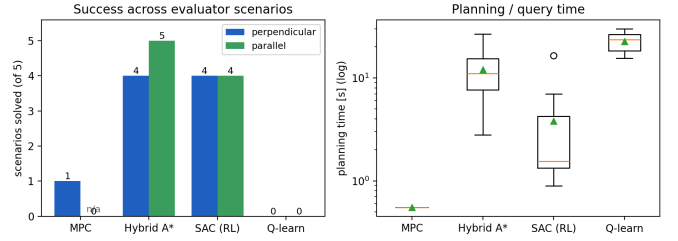


Figure 11: Cross-planner benchmark over the evaluator’s ten standard cases (two parking modes $\times$ five scenarios, one run each). Left: scenarios solved per mode. Right: planning / query time (log scale; for SAC this includes model loading, for Q-learning per-query training).

solves 9/10 cases, failing only the tight-lane perpendicular case, and is the most dependable planner in this benchmark. The deployed RL policies solve 8/10 cases and are faster at query time (median 1.6 s including model loading, versus 11 s for Hybrid A*), but this should be interpreted as deployment-time performance on these specific layouts rather than broad obstacle-avoidance capability. Several named obstacle layouts lie outside the learned maneuver corridor, so RL can succeed on them without executing a full obstacle detour; this does not contradict the fixed-start obstacle limitation shown in Figure 9. The evaluator also exercises the MPC reference only on the obstacle-free perpendicular case, while its parallel behavior is shown qualitatively in Figure 5. Tabular Q-learning solves none of the cases, with the highest per-query cost because it retrains at every call.

5.2 TRUCK-TRAILER SIMULATION

We evaluate the tractor-trailer planner on the two scenes described in Section 3: perpendicular docking and parallel parking. Both maneuvers are planned by the Hybrid A* search of Section 2.2 extended to the trailer-centric 4-dimensional state space, and replayed in the interactive simulator. In all figures, the truck body is drawn in blue and the trailer in green; the dashed yellow rectangle marks the target parking slot. The ghost outlines overlaid along the path show sampled poses of the tractor-trailer system at regular intervals during the maneuver.

5.2.1 PERPENDICULAR DOCKING

Figure 12 (top row) shows three snapshots of the perpendicular parking maneuver. Starting from a pose east of the slot and heading east (Figure 12a), the planner computes a path that swings the tractor past the slot entrance and then reverses the entire rig southward into the vertical slot. The mid-maneuver snapshot (Figure 12b) captures the critical phase: the tractor has already pivoted across the slot opening while the trailer, driven by the hitch coupling, begins to follow the planned arc into the slot. The hitch angle $\Delta\psi$ grows substantially during this phase but remains within the enforced limit of 55° . In the final state

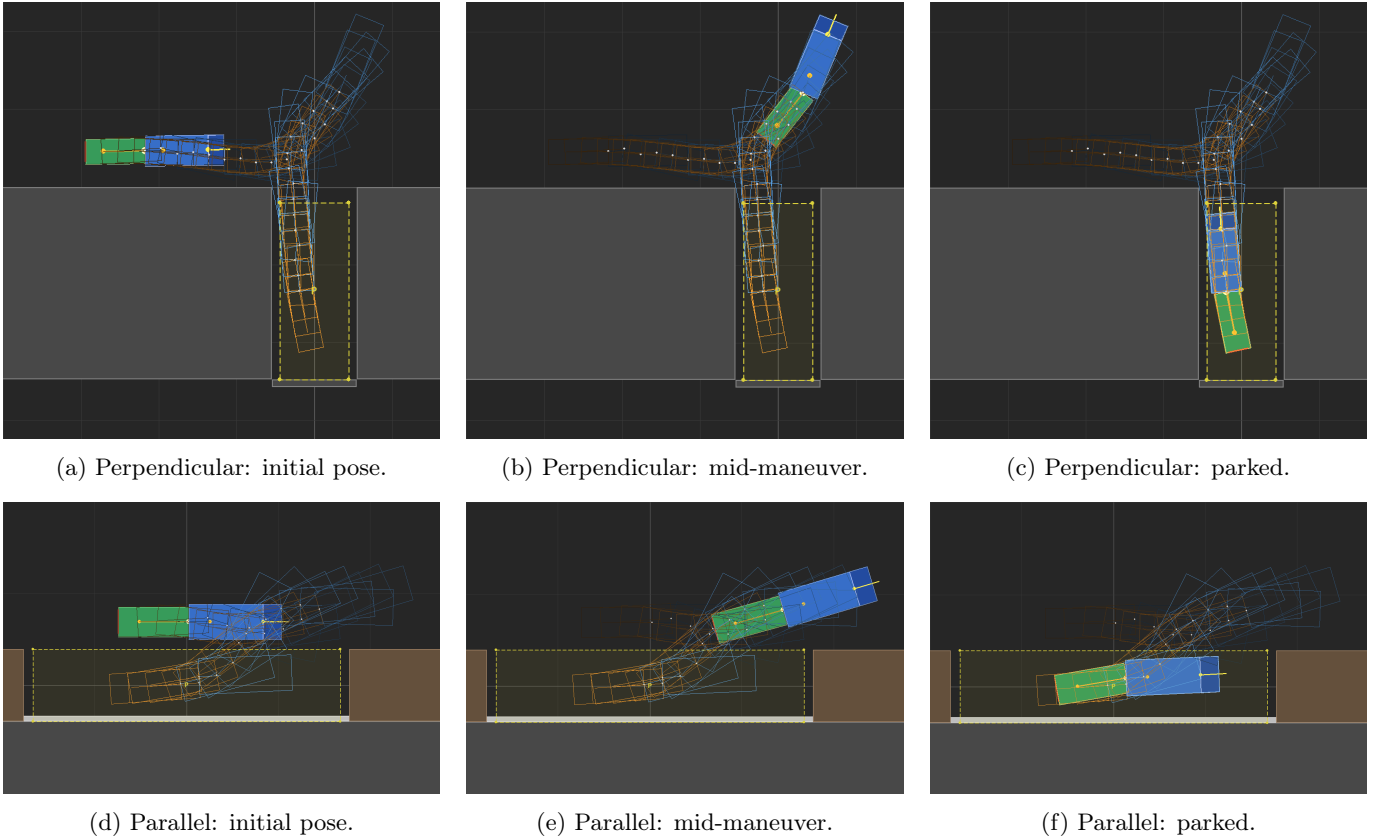


Figure 12: Tractor-trailer simulation results. **Top row:** perpendicular parking — the rig reverses into the vertical slot from an eastward starting pose. **Bottom row:** parallel parking — an extension beyond the original paper’s scope, with the slot flanked by kerb walls on both sides. Blue body = truck; green body = trailer; dashed yellow rectangle = target slot; ghost outlines show intermediate system poses along the planned path.

(Figure 12c), both the truck and trailer are fully inside the slot with the trailer heading aligned to $\psi_2 \approx +90^\circ$ and the hitch angle returned close to zero, satisfying the goal condition.

5.2.2 PARALLEL PARKING

Figure 12 (bottom row) shows the corresponding snapshots for the parallel parking scene, which is not covered in the original reference paper [3] and constitutes an extension of the reproduced method. The slot is oriented horizontally and flanked by kerb walls on both sides, making the maneuver considerably harder: the full tractor-trailer length must be threaded into the slot sideways, requiring a larger hitch angle excursion than in the perpendicular case. Starting from an eastward pose to the right of the slot (Figure 12d), the planner steers the tractor sharply left while reversing, building up a large hitch angle to sweep the trailer into the opening. The mid-maneuver snapshot (Figure 12e) shows the tractor already angled well into the slot while the trailer is still entering from the right; the swept envelope of the two bodies spans nearly the full slot width at this point. By the final state (Figure 12f), both units are centered inside the slot with $\psi_2 \approx 0$ and $\Delta\psi \approx 0$, and the additional corner check confirms that all

eight body corners lie within the slot boundary.

Both scenes confirm that the trailer-aware Hybrid A* planner successfully plans kinematically feasible, collision-free paths for the full tractor-trailer system in reverse motion, with the hitch angle remaining within safe limits throughout. The parallel scene additionally demonstrates that the method generalizes beyond the perpendicular docking task studied in the reference paper.

6 CONCLUSION

We presented an implementation study of autonomous parking motion planning. Within a single interactive system we built and compared three planner families for an ordinary car, a geometric MPC planner, a search-based Hybrid A*, and a learning-based RL planner, across both perpendicular and parallel (curb-side) parking, and we reproduced and extended a Hybrid A*-based tractor-trailer reverse-parking method, additionally solving a parallel scene not studied in the original work. Throughout, we assumed a known scene and focused on the planning problem.

Our experiments reveal a clear trade-off. MPC produces the smoothest and fastest maneuvers, but it is brittle, succeeding mainly when the space is generous and

obstacle-free. Hybrid A* is the most dependable stand-alone planner: it handles tight geometry, both parking modes, and user-placed obstacles, at the cost of less regular paths and higher computation. RL was initially the least reliable, with 0% success from the fixed deployment start under our first reward design, but became competitive after being scaffolded by a Hybrid A* reference path, demonstration seeding, and a reverse curriculum. The final policies park from the fixed start in both parking modes: pure SAC for perpendicular parking, and residual SAC over a pure-pursuit base controller for parallel parking. With domain randomization, the perpendicular policy further handles about half of the randomized scenes and feasibility-checked single-obstacle scenarios, usually failing safely by stopping when it does not succeed. However, its obstacle competence remains partial: it appears mainly under the randomized-start evaluation protocol, and the policy does not yet complete a full fixed-start obstacle detour. We also note that a simple Pure-Pursuit tracker following the Hybrid A* reference is a strong baseline on its own, parking reliably from the fixed start; much of the difficulty therefore lies in planning a good reference rather than in tracking it. Overall, search-based planning remains the most practical stand-alone choice, while our RL results suggest that learning-based planners are most effective when scaffolded by classical planning components such as reference paths, demonstrations, curricula, feasibility checks, and base controllers.

We show that the trailer-aware Hybrid A* planner successfully plans kinematically feasible, collision-free paths for the tractor-trailer system in both the perpendicular docking and the parallel parking scenes (Section 5.2), with the hitch angle remaining within safe limits throughout both maneuvers. The parallel scene, which is not studied in the original paper [3], confirms that the inverse-kinematics reparameterization and the adaptive δ_T range generalize beyond the perpendicular docking task. Compared to the single-vehicle planner, the trailer search is heavier (no analytic shot, weaker heuristic, 4-D state space) and runs in a background thread to keep the simulator responsive; a stronger nonholonomic heuristic and a trailer-specific analytic shot are natural directions for future work.

The study is limited in scope: we assume a known scene and do not address perception, and the quantitative cross-planner benchmark (Section 5.1.5) is modest, ten deterministic cases on the default scene, without start-pose or size randomization. The RL planners in particular generalize only partially across scene sizes, cannot solve the tightest lanes, and handle a single static obstacle (Section 5.1.4, Limitations). The tractor-trailer planner likewise has not been evaluated under randomized scene sizes or obstacle configurations. Natural next steps are scene-conditioned policies or per-scene fine-tuning, a closed-loop tracking controller, a larger randomized benchmark across vehicle and lot sizes, and a connection to a higher-fidelity simulator (e.g., CARLA) and, eventually, perception.

REFERENCES

- [1] Pandas Team, *Automatic Parallel Parking: Path Planning, Path Tracking and Control*. <https://github.com/Pandas-Team/Automatic-Parking>
- [2] A. Sakai, D. Ingram, J. Dinius, K. Chawla, A. Raffin, and A. Paques, “PythonRobotics: A Python Code Collection of Robotics Algorithms,” *arXiv:1808.10703*, 2018. <https://arxiv.org/abs/1808.10703>
- [3] X. Cao, H. Chen, B. Aksun-Guvenc, L. Guvenc, B. Link, P. J. Richmond, D. Yim, S. Fan, and J. Harber, “Hybrid A*-Based Reverse Path-Planning of a Vehicle with Single Trailer,” *Electronics*, vol. 15, no. 5, article 1114, 2026. <https://doi.org/10.3390/electronics15051114>
- [4] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, PMLR, vol. 80, pp. 1861–1870, 2018. <https://proceedings.mlr.press/v80/haarnoja18b.html>
- [5] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing Function Approximation Error in Actor-Critic Methods,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, PMLR, vol. 80, pp. 1587–1596, 2018. <https://proceedings.mlr.press/v80/fujimoto18a.html>
- [6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017. <https://arxiv.org/abs/1707.06347>
- [7] E. Leurent, “An Environment for Autonomous Driving Decision-Making,” GitHub repository, 2018. <https://github.com/eleurent/highway-env>
- [8] S. Ross, G. Gordon, and D. Bagnell, “A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning,” in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, PMLR, vol. 15, pp. 627–635, 2011. <https://proceedings.mlr.press/v15/ross11a.html>
- [9] C. J. C. H. Watkins and P. Dayan, “Q-Learning,” *Machine Learning*, vol. 8, pp. 279–292, 1992. <https://doi.org/10.1007/BF00992698>
- [10] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018. <https://mitpress.mit.edu/9780262039246/reinforcement-learning/>

-
- [11] M. Towers et al., “Gymnasium: A Standard Interface for Reinforcement Learning Environments,” *arXiv:2407.17032*, 2024. <https://arxiv.org/abs/2407.17032>
 - [12] A. Y. Ng, D. Harada, and S. Russell, “Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping,” in *Proceedings of the 16th International Conference on Machine Learning (ICML)*, pp. 278–287, 1999. <https://dl.acm.org/doi/10.5555/645528.657613>
 - [13] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, “Residual Reinforcement Learning for Robot Control,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6023–6029, 2019. <https://doi.org/10.1109/ICRA.2019.8794127>
 - [14] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, “Path Planning for Autonomous Vehicles in Unknown Semi-structured Environments,” *The International Journal of Robotics Research*, vol. 29, no. 5, pp. 485–501, 2010. <https://doi.org/10.1177/0278364909359210>
 - [15] J. A. Reeds and L. A. Shepp, “Optimal Paths for a Car That Goes Both Forwards and Backwards,” *Pacific Journal of Mathematics*, vol. 145, no. 2, pp. 367–393, 1990. <https://doi.org/10.2140/pjm.1990.145.367>
 - [16] P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968. <https://doi.org/10.1109/TSSC.1968.300136>
 - [17] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, and W. Zaremba, “Hindsight Experience Replay,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. <https://papers.nips.cc/paper/7090-hindsight-experience-replay>
 - [18] D. Gonz’alez, J. P’erez, V. Milan’es, and F. Nashashibi, “A Review of Motion Planning Techniques for Automated Vehicles,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 17, no. 4, pp. 1135–1145, 2016. <https://doi.org/10.1109/TITS.2015.2498841>
 - [19] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Nob Hill Publishing, 2017. <https://sites.engineering.ucsb.edu/~jbraw/mpc/>
 - [20] M. Montemerlo et al., “Junior: The Stanford Entry in the Urban Challenge,” *Journal of Field Robotics*, vol. 25, no. 9, pp. 569–597, 2008. <https://doi.org/10.1002/rob.20258>